Laporan Tugas Kecil 2

Strategi Algoritma (IF2211)

Disusun Oleh:

Mirza Tsabita Wafa'Ana (13524114)

Ahmad Rinofaros Muchtar (13524138)

Program Studi Teknik Informatika

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

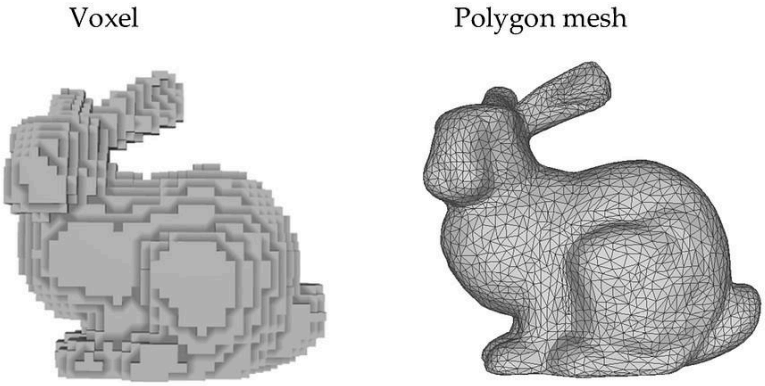
2025

DAFTAR ISI

DAFTAR ISI **2**
BAB 1 DESKRIPSI TUGAS **3**
BAB 2 PENYELESAIAN ALGORITMA **5**
2.2. Implementasi 7
2.3. Source Code 8
BAB 3 SCREENSHOT INPUT OUTPUT **18**
3.1. TestCase 1 18
3.2. TestCase 2 18
3.4. TestCase 4 18
3.5. TestCase 5 19
BAB 4 LAMPIRAN **20**
4.1. Link Repository Program 20
4.2. Pernyataan Tidak Melakukan Kecurangan 20
Daftar Pustaka 21

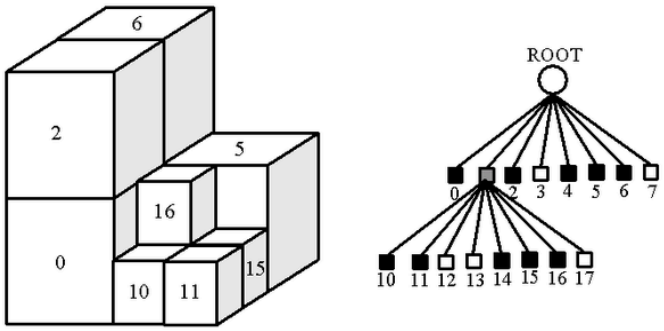
BAB 1 DESKRIPSI TUGAS

Deskripsi Tugas



Gambar 1 Visualisasi *voxelization*
Sumber: https://www.mdpi.com/1424-8220/22/18/7040

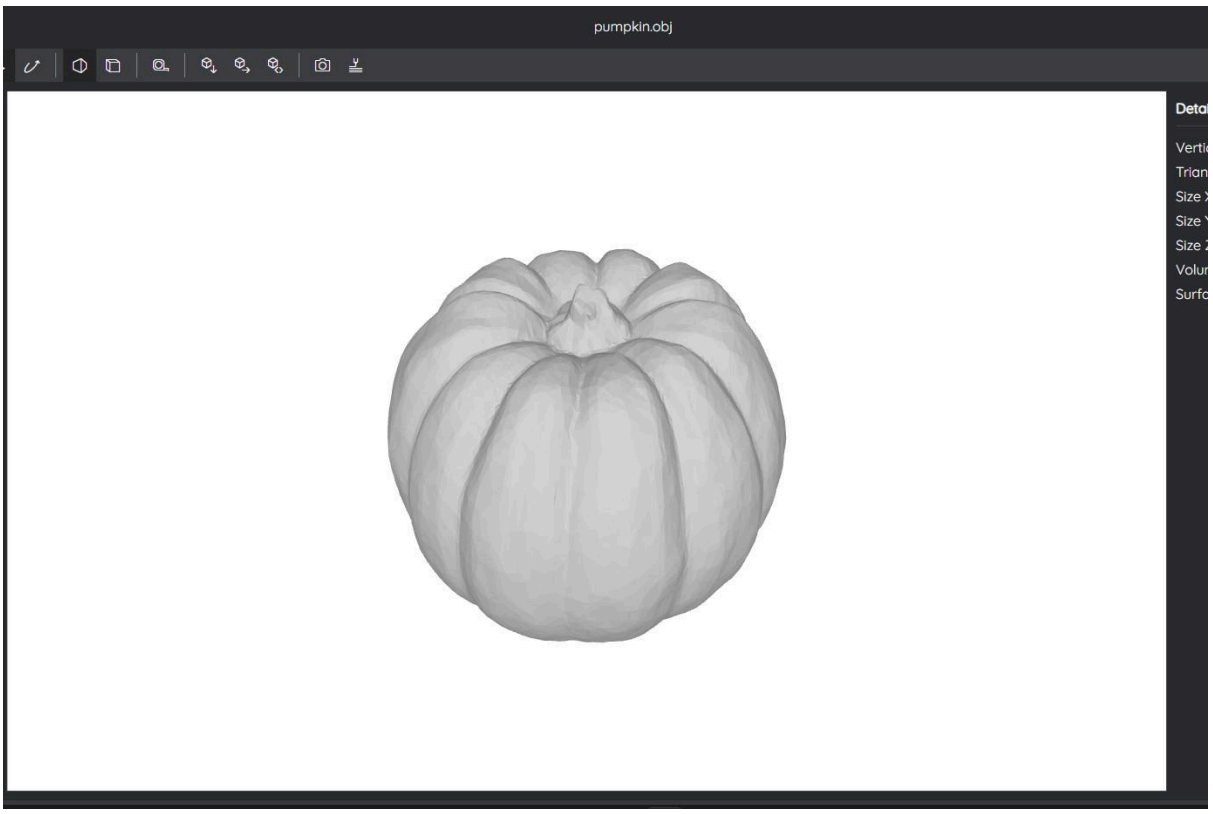
Voxelization merupakan proses konversi objek tiga dimensi yang umum menjadi sebuah model yang tersusun dari kubus-kubus kecil seperti di Minecraft. Proses konversi ini dilakukan dengan memanfaatkan struktur data **Octree**, yaitu sebuah *tree* yang setiap percabangan terdiri dari 8 anak. Melalui *Octree*, sebuah ruang kubus 3D dapat dipotong menjadi 8 kubus berbeda, yang masing-masing dapat dipotong lagi menjadi 8 kubus yang lebih kecil.



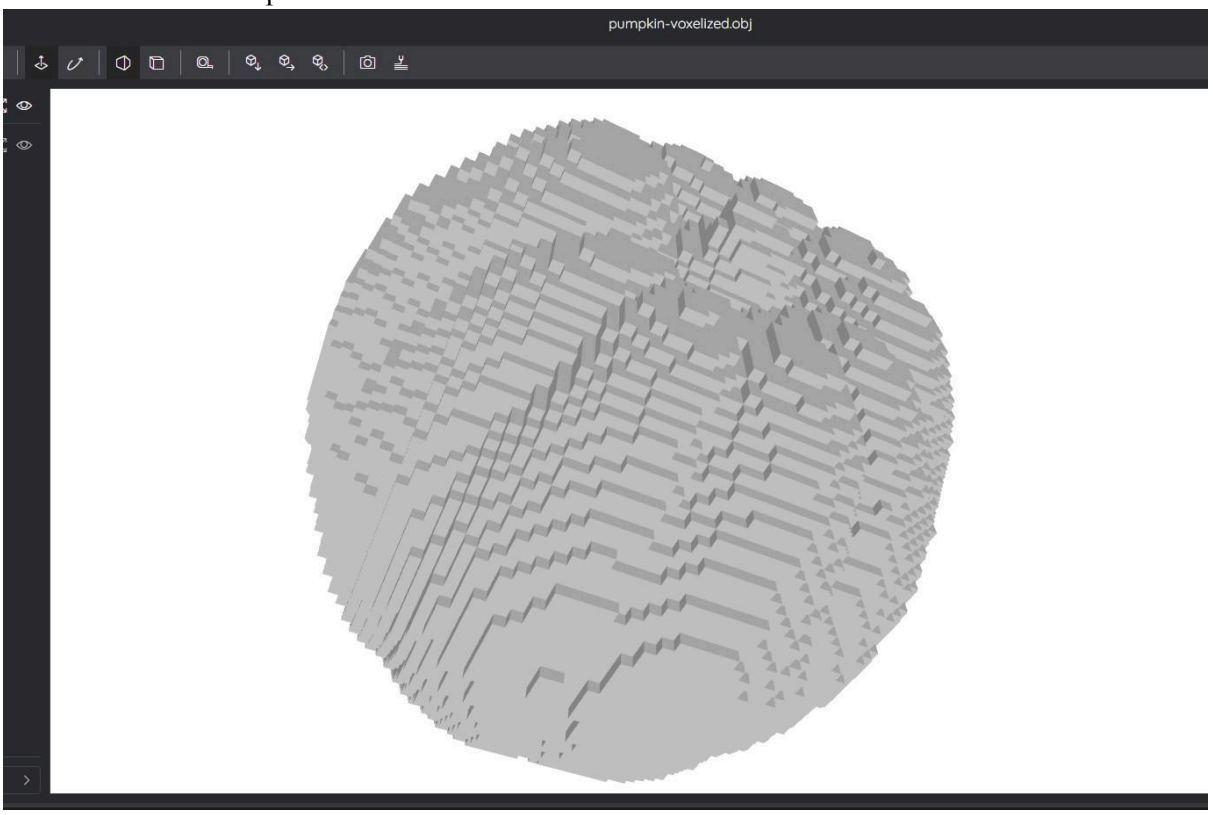
Gambar 2 Visualisasi *Octree* terhadap 3D Model
Sumber:
https://www.researchgate.net/figure/a-Octants-b-Octree-Structure-c-Linear-Octree-Each-of-the-cubes-has-a-locational-code_fig1_221115507

Tugas Anda adalah mengimplementasikan sebuah program yang dapat mengkonversi model .obj yang umum menjadi model .obj yang terdiri dari *voxel* dengan memanfaatkan properti ***Octree***. Program melakukan konversi model dengan menggunakan algoritma ***divide and conquer.***

Ilustrasi Kasus
Diberikan sebuah model 3D sebagai berikut.



Di bawah adalah hasil perubahan.



BAB 2 PENYELESAIAN ALGORITMA

2.1. Algoritma yang Digunakan

Pada tugas ini, proses **voxelization objek 3D** dilakukan menggunakan algoritma **divide and conquer** berbasis struktur data **Octree**. Program bekerja dengan membaca file .obj, mengambil data vertex dan face, lalu membentuk representasi segitiga dari setiap face. Setelah itu, program menghitung bounding box dari seluruh vertex dan mengubahnya menjadi sebuah kubus sebagai root Octree. Proses pembagian ruang dilakukan secara rekursif dengan membagi setiap node menjadi 8 subruang. Pada setiap node, program memeriksa apakah ada segitiga yang beririsan dengan bounding box node tersebut. Jika tidak ada irisan, node langsung dihentikan. Jika ada irisan dan kedalaman maksimum belum tercapai, node dibagi lagi. Jika kedalaman maksimum sudah tercapai, node tersebut ditandai sebagai voxel terisi.

Algoritma yang diimplementasikan tidak langsung membagi seluruh ruang secara penuh, tetapi hanya membagi node yang masih berpotensi memuat geometri objek. Untuk menentukan hal tersebut, program menggunakan fungsi triAABBIntersect() yang menguji perpotongan antara segitiga dan bounding box node dengan pendekatan **Separating Axis Theorem (SAT)**. Dengan demikian, proses divide and conquer dilakukan secara selektif: node kosong akan dipangkas, sedangkan node yang beririsan akan terus diproses hingga mencapai depth maksimum. Setelah seluruh proses selesai, semua leaf node yang terisi dikumpulkan sebagai voxel, lalu dituliskan kembali menjadi file .obj keluaran. Hasil akhirnya sesuai dengan syarat tugas, yaitu menghasilkan model voxel berukuran seragam pada depth maksimum dan menampilkan statistik proses di CLI.

Langkah-langkah algoritma yang digunakan adalah sebagai berikut:

1. **Pembacaan dan Validasi Input**
   Program meminta pengguna memasukkan dua argumen pada command line, yaitu input.obj dan max_depth. Jika jumlah argumen kurang dari 2 atau nilai max_depth bukan bilangan bulat positif, program langsung menampilkan pesan kesalahan dan berhenti. Setelah itu, file .obj dibuka menggunakan fungsi parseOBJ().
2. **Parsing File OBJ**
   Program membaca file .obj baris per baris menggunakan bufio.Scanner. Baris kosong dan komentar yang diawali # dilewati. Jika token pertama adalah v, program membaca tiga nilai koordinat x, y, dan z, lalu menyimpannya sebagai Vec3. Jika token pertama adalah f, program membaca indeks-indeks vertex pada face tersebut. Karena face dapat memiliki lebih dari tiga titik, program melakukan **triangulasi fan**, yaitu membentuk beberapa Triangle dari satu face dengan pola titik pertama sebagai acuan. Dengan cara ini, semua face pada input diubah menjadi kumpulan segitiga. Jika tidak ada vertex atau tidak ada face yang berhasil dibaca, file dianggap tidak valid dan program dihentikan.
3. **Perhitungan Bounding Box Awal**
   Setelah seluruh vertex berhasil dibaca, program memanggil fungsi computeBounds() untuk mencari nilai minimum dan maksimum pada sumbu x, y, dan z. Dari hasil tersebut, program tidak langsung memakai box biasa, tetapi memilih panjang sisi terbesar dari ketiga dimensi dan membentuk **bounding box kubus** yang terpusat di tengah objek. Sisi kubus ini sedikit diperbesar dengan faktor 1.001 agar seluruh objek benar-benar masuk ke dalam root box.
4. **Inisialisasi Struktur Octree**
   Program membuat root node bertipe OctreeNode yang berisi bounding box hasil computeBounds(). Selain itu, program juga menyiapkan struktur statistik BuildStats yang menyimpan:
   a. jumlah node yang dibangun pada tiap depth,
   b. jumlah node yang dipruning pada tiap depth,

c. dan nilai depth maksimum.
   Setelah itu, program memanggil fungsi subdivide() untuk mulai membangun Octree secara rekursif dari root.

5. **Penyaringan Segitiga pada Setiap Node**
   Di dalam fungsi subdivide(), program terlebih dahulu menambah penghitung jumlah node pada depth saat itu. Lalu, program memeriksa semua segitiga yang diberikan ke node tersebut satu per satu. Untuk setiap segitiga, program memanggil fungsi triAABBIntersect() untuk mengecek apakah segitiga beririsan dengan bounding box node. Semua segitiga yang beririsan disimpan dalam array hitting.
6. **Pemangkasan Node Kosong**
   Jika setelah pemeriksaan ternyata hitting kosong, berarti tidak ada geometri yang melewati node tersebut. Dalam kondisi ini, node langsung dijadikan leaf, IsFilled diatur false, dan penghitung PrunedPerDepth pada depth tersebut ditambah satu. Node seperti ini tidak dibagi lagi dan proses rekursi dihentikan pada cabang tersebut.
7. **Penandaan Leaf Terisi**
   Jika hitting tidak kosong dan depth saat ini sudah sama dengan maxDepth, program tidak membagi node lagi. Node langsung dijadikan leaf dengan IsFilled = true. Node inilah yang nantinya dianggap sebagai voxel aktif pada hasil akhir.
8. **Pembagian Node Menjadi 8 Subnode**
   Jika hitting tidak kosong dan depth maksimum belum tercapai, program membagi bounding box node menjadi 8 sub-box. Pembagian dilakukan dengan menghitung titik pusat box saat ini, lalu membentuk 8 kombinasi ruang dari nilai minimum, titik tengah, dan maksimum pada sumbu x, y, dan z. Untuk setiap sub-box, program membuat child node baru dan memanggil subdivide() lagi secara rekursif. Yang penting, child hanya menerima daftar segitiga hitting, bukan seluruh segitiga awal, sehingga proses pada level berikutnya menjadi lebih terfokus.
9. **Pemeriksaan Interseksi Segitiga–AABB**
   Fungsi triAABBIntersect() menguji apakah sebuah segitiga berpotongan dengan sebuah bounding box menggunakan **Separating Axis Theorem (SAT)**. Langkah yang dilakukan adalah:
   a. menghitung pusat dan half-size dari box,
   b. mentranslasikan titik-titik segitiga agar relatif terhadap pusat box,
   c. menghitung tiga edge segitiga,
   d. menguji sumbu-sumbu hasil cross product antara edge segitiga dan sumbu utama box,
   e. menguji tiga normal sisi AABB,
   f. dan menguji normal bidang segitiga.
      Jika pada salah satu sumbu ditemukan pemisahan, fungsi langsung mengembalikan false. Jika semua pengujian lolos, fungsi mengembalikan true, artinya segitiga dan box saling beririsan.
10. **Pengumpulan Voxel Hasil**
    Setelah seluruh proses subdivisi selesai, program memanggil collectVoxels() untuk menelusuri Octree dan mengumpulkan semua leaf node yang memiliki IsFilled = true. Fungsi ini bekerja secara rekursif: jika node adalah leaf dan terisi, bounding box node dimasukkan ke hasil; jika node bukan leaf, fungsi dipanggil ke seluruh child node.
11. **Pembentukan File Output OBJ**
    Setelah semua voxel terkumpul, program membuat nama file keluaran dengan format namaFile_voxelized_depthN.obj. Selanjutnya, fungsi writeVoxelOBJ() dipanggil untuk menuliskan seluruh voxel ke file .obj. Untuk setiap voxel, program:
    a. mengambil nilai minimum dan maksimum bounding box,
    b. membentuk 8 titik sudut kubus,
    c. menuliskan 8 vertex ke file,
    d. lalu membentuk 6 face berbentuk quad.

    Program juga menjaga vertexOffset agar indeks vertex antarvoxel tidak saling bertabrakan. Jumlah total vertex dan face dihitung selama proses ini.

12. **Output Statistik dan Terminasi**
    Setelah file berhasil ditulis, program menghitung lama eksekusi menggunakan time.Since(start). Lalu program menampilkan informasi ke CLI, yaitu:
    - jumlah voxel terisi,
    - jumlah vertex output,
    - jumlah face output,
    - statistik node Octree per depth,
    - statistik node yang dipruning per depth,
    - kedalaman Octree,
    - waktu eksekusi,
    - dan path file output.

    Setelah seluruh informasi ditampilkan, program selesai dijalankan.

2.2. Implementasi

Nama	Jenis	Deskripsi
parseOBJ(path)	Fungsi	Membaca file .obj, mengambil vertex (v) dan face (f), serta mengubah face menjadi segitiga. Mengembalikan data geometri atau error jika tidak valid.
computeBounds(vertices)	Fungsi	Menghitung bounding box dari seluruh vertex, lalu mengubahnya menjadi kubus sebagai root Octree.
subdivide(node, triangles, depth, maxDepth, stats)	Prosedur	Membangun Octree secara rekursif. Node tanpa irisan dipruning, node pada depth maksimum ditandai sebagai voxel, lainnya dibagi menjadi 8 child.
triAABBIntersect(tri, box)	Fungsi	Mengecek apakah segitiga beririsan dengan bounding box menggunakan metode SAT.
collectVoxels(node)	Fungsi	Mengambil semua leaf node yang terisi sebagai voxel hasil.
writeVoxelOBJ(voxels, path)	Fungsi	Menulis voxel ke file .obj sebagai kubus dengan vertex dan face.
BuildStats	Struktur	Menyimpan statistik node per depth dan jumlah node yang dipruning.
main()	Prosedur	Alur utama: baca input → parse → hitung bounding box → bangun Octree → kumpulkan voxel → simpan output → tampilkan statistik.

2.3. Source Code

File go.mod

```
module voxelizer

go 1.22.2
```

File main.go

```
package main

import (
	"bufio"
	"fmt"
	"math"
	"os"
	"path/filepath"
	"strconv"
	"strings"
	"time"
)

// Data types

type Vec3 struct{ X, Y, Z float64 }

type Triangle struct{ A, B, C Vec3 }

type AABB struct {
	Min, Max Vec3
}

func (b AABB) Center() Vec3 {
	return Vec3{
		(b.Min.X + b.Max.X) / 2,
		(b.Min.Y + b.Max.Y) / 2,
		(b.Min.Z + b.Max.Z) / 2,
	}
}

func (b AABB) Size() Vec3 {
	return Vec3{b.Max.X - b.Min.X, b.Max.Y - b.Min.Y, b.Max.Z - b.Min.Z}
}
```

```
// OBJ parsing

func parseOBJ(path string) ([]Vec3, []Triangle, error) {
	f, err := os.Open(path)
	if err != nil {
		return nil, nil, fmt.Errorf("Gagal Buka File: %w", err)
	}
	defer f.Close()

	var vertices []Vec3
	var triangles []Triangle
	lineNum := 0

	scanner := bufio.NewScanner(f)
	for scanner.Scan() {
		lineNum++
		line := strings.TrimSpace(scanner.Text())
		if line == "" || strings.HasPrefix(line, "#") {
			continue
		}
		fields := strings.Fields(line)
		if len(fields) == 0 {
			continue
		}

		switch fields[0] {
		case "v":
			if len(fields) < 4 {
				return nil, nil, fmt.Errorf("invalid vertex on line %d: %q",
lineNum, line)
			}
			x, err1 := strconv.ParseFloat(fields[1], 64)
			y, err2 := strconv.ParseFloat(fields[2], 64)
			z, err3 := strconv.ParseFloat(fields[3], 64)
			if err1 != nil || err2 != nil || err3 != nil {
				return nil, nil, fmt.Errorf("invalid vertex coordinates on line
%d: %q", lineNum, line)
			}
			vertices = append(vertices, Vec3{x, y, z})

		case "f":
			if len(fields) < 4 {
				return nil, nil, fmt.Errorf("invalid face on line %d (need ≥3
indices): %q", lineNum, line)
			}
			//  Mengolah tiap index
			//	format face hanya sebagai integer index
			indices := make([]int, 0, len(fields)-1)
```

```
                        for _, tok := range fields[1:] {
                                //  Ambil bagian pertama saja
                                parts := strings.Split(tok, "/")
                                idx, err := strconv.Atoi(parts[0])
                                if err != nil {
                                        return nil, nil, fmt.Errorf("invalid face index on line
%d: %q", lineNum, tok)
                                }
                                if idx < 1 || idx > len(vertices) {
                                        return nil, nil, fmt.Errorf("face index %d out of range
on line %d", idx, lineNum)
                                }
                                indices = append(indices, idx-1)
                        }
                        //
                        for i := 1; i < len(indices)-1; i++ {
                                triangles = append(triangles, Triangle{
                                        A: vertices[indices[0]],
                                        B: vertices[indices[i]],
                                        C: vertices[indices[i+1]],
                                })
                        }

                default:
                }
        }

        if err := scanner.Err(); err != nil {
                return nil, nil, err
        }
        if len(vertices) == 0 {
                return nil, nil, fmt.Errorf("File OBJ tidak valid, tidak ada vertex")
        }
        if len(triangles) == 0 {
                return nil, nil, fmt.Errorf("File OBJ tidak falid, tidak ada face")
        }
        return vertices, triangles, nil
}

// Bounding box

func computeBounds(vertices []Vec3) AABB {
        mn := Vec3{math.MaxFloat64, math.MaxFloat64, math.MaxFloat64}
        mx := Vec3{-math.MaxFloat64, -math.MaxFloat64, -math.MaxFloat64}
        for _, v := range vertices {
                if v.X < mn.X { mn.X = v.X }
                if v.Y < mn.Y { mn.Y = v.Y }
                if v.Z < mn.Z { mn.Z = v.Z }
```

```
		if v.X > mx.X { mx.X = v.X }
		if v.Y > mx.Y { mx.Y = v.Y }
		if v.Z > mx.Z { mx.Z = v.Z }
	}

	// Vektor diubah menjadi kubus
	size := mn
	_ = size
	sx := mx.X - mn.X
	sy := mx.Y - mn.Y
	sz := mx.Z - mn.Z
	side := math.Max(sx, math.Max(sy, sz))

	//  Vektor ditengahkan dan dilebarkan
	cx := (mn.X + mx.X) / 2
	cy := (mn.Y + mx.Y) / 2
	cz := (mn.Z + mx.Z) / 2
	half := side / 2 * 1.001 // tambahan
	return AABB{
		Min: Vec3{cx - half, cy - half, cz - half},
		Max: Vec3{cx + half, cy + half, cz + half},
	}
}

// Triangle–AABB intersection (SAT)

func triAABBIntersect(tri Triangle, box AABB) bool {
	center := box.Center()
	half := Vec3{
		(box.Max.X - box.Min.X) / 2,
		(box.Max.Y - box.Min.Y) / 2,
		(box.Max.Z - box.Min.Z) / 2,
	}

	//  Mengubah segitiga menjadi kotak di tengah
	v0 := Vec3{tri.A.X - center.X, tri.A.Y - center.Y, tri.A.Z - center.Z}
	v1 := Vec3{tri.B.X - center.X, tri.B.Y - center.Y, tri.B.Z - center.Z}
	v2 := Vec3{tri.C.X - center.X, tri.C.Y - center.Y, tri.C.Z - center.Z}

	//  Menyatakan edges
	e0 := Vec3{v1.X - v0.X, v1.Y - v0.Y, v1.Z - v0.Z}
	e1 := Vec3{v2.X - v1.X, v2.Y - v1.Y, v2.Z - v1.Z}
	e2 := Vec3{v0.X - v2.X, v0.Y - v2.Y, v0.Z - v2.Z}

	//  SAT: Menguji cross-product vektor dan edges
	axes := [][2]Vec3{
		{{1, 0, 0}, e0}, {{0, 1, 0}, e0}, {{0, 0, 1}, e0},
		{{1, 0, 0}, e1}, {{0, 1, 0}, e1}, {{0, 0, 1}, e1},
```

```
		{{1, 0, 0}, e2}, {{0, 1, 0}, e2}, {{0, 0, 1}, e2},
	}
	for _, pair := range axes {
		a := cross(pair[0], pair[1])
		p0 := dot(a, v0)
		p1 := dot(a, v1)
		p2 := dot(a, v2)
		r := half.X*math.Abs(a.X) + half.Y*math.Abs(a.Y) + half.Z*math.Abs(a.Z)
		mn := min3(p0, p1, p2)
		mx := max3(p0, p1, p2)
		if mn > r || mx < -r {
			return false
		}
	}

	//  SAT: 3 AABB face normals
	if max3(v0.X, v1.X, v2.X) < -half.X || min3(v0.X, v1.X, v2.X) > half.X { return false
}
	if max3(v0.Y, v1.Y, v2.Y) < -half.Y || min3(v0.Y, v1.Y, v2.Y) > half.Y { return false }
	if max3(v0.Z, v1.Z, v2.Z) < -half.Z || min3(v0.Z, v1.Z, v2.Z) > half.Z { return false }

	//  SAT: triangle face normal
	normal := cross(e0, e1)
	d := dot(normal, v0)
	r2 := half.X*math.Abs(normal.X) + half.Y*math.Abs(normal.Y) +
half.Z*math.Abs(normal.Z)
	if d > r2 || d < -r2 { return false }

	return true
}

func cross(a, b Vec3) Vec3 {
	return Vec3{
		a.Y*b.Z - a.Z*b.Y,
		a.Z*b.X - a.X*b.Z,
		a.X*b.Y - a.Y*b.X,
	}
}
func dot(a, b Vec3) float64 { return a.X*b.X + a.Y*b.Y + a.Z*b.Z }
func min3(a, b, c float64) float64 { return math.Min(a, math.Min(b, c)) }
func max3(a, b, c float64) float64 { return math.Max(a, math.Max(b, c)) }

// Octree

type OctreeNode struct {
	Bounds   AABB
	Children [8]*OctreeNode
	IsLeaf   bool
```

```
	IsFilled bool //  Leaf yang saling bertepatan menjadi voxel
}

//  Stats
type BuildStats struct {
	NodesPerDepth   []int
	PrunedPerDepth  []int
	MaxDepth        int
}

func buildOctree(bounds AABB, triangles []Triangle, maxDepth int) (*OctreeNode,
BuildStats) {
	stats := BuildStats{
		NodesPerDepth:  make([]int, maxDepth+1),
		PrunedPerDepth: make([]int, maxDepth+1),
		MaxDepth:       maxDepth,
	}
	root := &OctreeNode{Bounds: bounds}
	subdivide(root, triangles, 0, maxDepth, &stats)
	return root, stats
}

func subdivide(node *OctreeNode, triangles []Triangle, depth, maxDepth int, stats
*BuildStats) {
	stats.NodesPerDepth[depth]++

	//  Menyaring segitiga yang berlintasan
	var hitting []Triangle
	for _, tri := range triangles {
		if triAABBIntersect(tri, node.Bounds) {
			hitting = append(hitting, tri)
		}
	}

	if len(hitting) == 0 {
		//  Buntu
		stats.PrunedPerDepth[depth]++
		node.IsLeaf = true
		node.IsFilled = false
		return
	}

	if depth == maxDepth {
		// Kedalaman sudah maksimal
		node.IsLeaf = true
		node.IsFilled = true
		return
	}
```

```
	//  Bagi menjadi 8 subdivisi
	c := node.Bounds.Center()
	mn := node.Bounds.Min
	mx := node.Bounds.Max

	subBounds := [8]AABB{
		{Vec3{mn.X, mn.Y, mn.Z}, Vec3{c.X, c.Y, c.Z}},
		{Vec3{c.X, mn.Y, mn.Z}, Vec3{mx.X, c.Y, c.Z}},
		{Vec3{mn.X, c.Y, mn.Z}, Vec3{c.X, mx.Y, c.Z}},
		{Vec3{c.X, c.Y, mn.Z}, Vec3{mx.X, mx.Y, c.Z}},
		{Vec3{mn.X, mn.Y, c.Z}, Vec3{c.X, c.Y, mx.Z}},
		{Vec3{c.X, mn.Y, c.Z}, Vec3{mx.X, c.Y, mx.Z}},
		{Vec3{mn.X, c.Y, c.Z}, Vec3{c.X, mx.Y, mx.Z}},
		{Vec3{c.X, c.Y, c.Z}, Vec3{mx.X, mx.Y, mx.Z}},
	}

	for i := 0; i < 8; i++ {
		child := &OctreeNode{Bounds: subBounds[i]}
		node.Children[i] = child
		subdivide(child, hitting, depth+1, maxDepth, stats)
	}
}

//  Mengumpulkan semua leaf nodes (voxels)
func collectVoxels(node *OctreeNode) []AABB {
	if node == nil {
		return nil
	}
	if node.IsLeaf {
		if node.IsFilled {
			return []AABB{node.Bounds}
		}
		return nil
	}
	var result []AABB
	for _, child := range node.Children {
		result = append(result, collectVoxels(child)...)
	}
	return result
}

// OBJ output

func writeVoxelOBJ(path string, voxels []AABB) (int, int, error) {
	f, err := os.Create(path)
	if err != nil {
		return 0, 0, err
```

```
	}
	defer f.Close()

	w := bufio.NewWriter(f)
	fmt.Fprintf(w, "# Voxelized OBJ — %d voxels\n", len(voxels))

	vertexOffset := 0
	totalFaces := 0

	//  Mengiterasi tiap voxel yang memiliki 8 vertex dan 6 faces
	for _, box := range voxels {
		mn := box.Min
		mx := box.Max

		//  8 vertex
		corners := [8]Vec3{
			{mn.X, mn.Y, mn.Z}, {mx.X, mn.Y, mn.Z},
			{mx.X, mx.Y, mn.Z}, {mn.X, mx.Y, mn.Z},
			{mn.X, mn.Y, mx.Z}, {mx.X, mn.Y, mx.Z},
			{mx.X, mx.Y, mx.Z}, {mn.X, mx.Y, mx.Z},
		}
		for _, v := range corners {
			fmt.Fprintf(w, "v %f %f %f\n", v.X, v.Y, v.Z)
		}

		//  6 faces as quads (i+offset, 1-based)
		o := vertexOffset
		faces := [6][4]int{
			{o + 1, o + 2, o + 3, o + 4}, //  bottom  (-Y)
			{o + 5, o + 8, o + 7, o + 6}, //  top     (+Y)
			{o + 1, o + 5, o + 6, o + 2}, //  front   (-Z)
			{o + 4, o + 3, o + 7, o + 8}, //  back    (+Z)
			{o + 1, o + 4, o + 8, o + 5}, //  left    (-X)
			{o + 2, o + 6, o + 7, o + 3}, //  right   (+X)
		}
		for _, face := range faces {
			fmt.Fprintf(w, "f %d %d %d %d\n", face[0], face[1], face[2], face[3])
			totalFaces++
		}

		vertexOffset += 8
	}

	return vertexOffset, totalFaces, w.Flush()
}

// main
```

```go
func main() {
	if len(os.Args) < 3 {
		fmt.Fprintln(os.Stderr, "Usage: voxelizer <input.obj> <max_depth>")
		os.Exit(1)
	}

	inputPath := os.Args[1]
	maxDepth, err := strconv.Atoi(os.Args[2])
	if err != nil || maxDepth < 1 {
		fmt.Fprintln(os.Stderr, "Error: max_depth must be a positive integer")
		os.Exit(1)
	}

	start := time.Now()

	// Parse
	vertices, triangles, err := parseOBJ(inputPath)
	if err != nil {
		fmt.Fprintf(os.Stderr, "Error parsing OBJ: %v\n", err)
		os.Exit(1)
	}

	// Octree Build
	bounds := computeBounds(vertices)
	root, stats := buildOctree(bounds, triangles, maxDepth)

	// Collect Voxels
	voxels := collectVoxels(root)

	// Print Output
	ext := filepath.Ext(inputPath)
	base := strings.TrimSuffix(inputPath, ext)
	outputPath := fmt.Sprintf("%s_voxelized_depth%d.obj", base, maxDepth)

	totalVertices, totalFaces, err := writeVoxelOBJ(outputPath, voxels)
	if err != nil {
		fmt.Fprintf(os.Stderr, "Gagal Print OBJ: %v\n", err)
		os.Exit(1)
	}

	elapsed := time.Since(start)

	// ── CLI stats ──────────────────────────────────────────────
	fmt.Printf("Voxels (filled leaf nodes) : %d\n", len(voxels))
	fmt.Printf("Vertices in output         : %d\n", totalVertices)
	fmt.Printf("Faces in output            : %d\n", totalFaces)
	fmt.Println()
	fmt.Println("Octree node stats (nodes built per depth):")
```

```
	for d := 0; d <= maxDepth; d++ {
		if stats.NodesPerDepth[d] > 0 {
			fmt.Printf("  %d : %d\n", d+1, stats.NodesPerDepth[d])
		}
	}
	fmt.Println()
	fmt.Println("Nodes pruned (no geometry) per depth:")
	for d := 0; d <= maxDepth; d++ {
		if stats.NodesPerDepth[d] > 0 {
			fmt.Printf("  %d : %d\n", d+1, stats.PrunedPerDepth[d])
		}
	}
	fmt.Println()
	fmt.Printf("Octree depth              : %d\n", maxDepth)
	fmt.Printf("Time elapsed              : %v\n", elapsed)
	fmt.Printf("Output file               : %s\n", outputPath)
}
```

BAB 3 SCREENSHOT INPUT OUTPUT

3.1. TestCase 1

Cow.obj
input:
https://drive.google.com/file/d/1pzNhYGtIYHlypH7sWzZeeahq_wx-pZbf/view?usp=sharing
output:
https://drive.google.com/file/d/1kEqakBa5Ibgl1_V-Vj7fHDBLPEJdNvWj/view?usp=drive_link
buildStats:

```
rino@LAPTOP-ESI19P0R:/mnt/d/Tucil2_13524138_13524137/src$ ./voxelizer /mnt/d/Tucil2_13524138_13524137/test/cow.obj 5
Voxels (filled leaf nodes) : 1362
Vertices in output         : 10896
Faces in output            : 8172

Octree node stats (nodes built per depth):
  1 : 1
  2 : 8
  3 : 64
  4 : 160
  5 : 704
  6 : 2656

Nodes pruned (no geometry) per depth:
  1 : 0
  2 : 0
  3 : 44
  4 : 72
  5 : 372
  6 : 1294

Octree depth               : 5
Time elapsed               : 118.646058ms
Output file                : /mnt/d/Tucil2_13524138_13524137/test/cow_voxelized_depth5.obj
```

3.2. TestCase 2

line.obj
input:
https://drive.google.com/file/d/1Qb4n5ruO0jCQdS-UsKa0MoxsLQlfzp-z/view?usp=drive_link
output:
https://drive.google.com/file/d/1Qb4n5ruO0jCQdS-UsKa0MoxsLQlfzp-z/view?usp=drive_link
buildStats:

```
rino@LAPTOP-ESI19P0R:/mnt/d/Tucil2_13524138_13524137/src$ go run /mnt/d/Tucil2_13524138_13524137/src/main.go  /mnt/d/Tucil2_13
524138_13524137/test/line.obj 5
Voxels (filled leaf nodes) : 286
Vertices in output         : 2288
Faces in output            : 1716

Octree node stats (nodes built per depth):
  1 : 1
  2 : 8
  3 : 56
  4 : 120
  5 : 264
  6 : 736

Nodes pruned (no geometry) per depth:
  1 : 0
  2 : 1
  3 : 41
  4 : 87
  5 : 172
  6 : 450

Octree depth               : 5
Time elapsed               : 30.960434ms
Output file                : /mnt/d/Tucil2_13524138_13524137/test/line_voxelized_depth5.obj
```

3.3. TestCase 3

Pumpkin.obj

input:

https://drive.google.com/file/d/1VTNS5hMREx09lEPa9LRa96IzxwlJ20Yq/view?usp=sharing

Output:

https://drive.google.com/file/d/1ocz3UgUqKuAYLouZM4_yJHqcR4jgihNd/view?usp=drive_link

buildStats:

```
rino@LAPTOP-ESI19P0R:/mnt/d/Tucil2_13524138_13524137/src$ go run /mnt/d/Tucil2_13524138_13524137/src/main.go  /mnt/d/Tucil2_13
524138_13524137/test/pumpkin.obj 5
Voxels (filled leaf nodes) : 4619
Vertices in output         : 36952
Faces in output            : 27714

Octree node stats (nodes built per depth):
  1 : 1
  2 : 8
  3 : 64
  4 : 448
  5 : 2216
  6 : 8968

  4 : 171
  4 : 171
  5 : 1095
  6 : 4349

Octree depth               : 5
Time elapsed               : 282.589325ms
```

3.4. TestCase 4

teapot.obj

input:

https://drive.google.com/file/d/1xP7HLEsjx3QDrarPQ6HLJ-21msDBJc8i/view?usp=sharing

output:

https://drive.google.com/file/d/1b0kCYw4JRoqrtBjcgtD5B1-Ua4Gkyfyq/view?usp=drive_link

buildStats:

```
rino@LAPTOP-ESI19P0R:/mnt/d/Tucil2_13524138_13524137/src$ go run /mnt/d/Tucil2_13524138_13524137/src/main.go  /mnt/d/Tucil2_13
524138_13524137/test/teapot.obj 5
Voxels (filled leaf nodes) : 1746
Vertices in output         : 13968
Faces in output            : 10476

Octree node stats (nodes built per depth):
  1 : 1
  2 : 8
  3 : 64
  4 : 192
  5 : 864
  6 : 3344

Nodes pruned (no geometry) per depth:
  1 : 0
  2 : 0
```

3.5. TestCase 5

Konten.obj

input:

https://drive.google.com/file/d/1hRd139E2ZjSoRSMh0RfmLVfYu1Kyu9tH/view?usp=drive_link

output:

https://drive.google.com/file/d/1kEqakBa5Ibgl1_V-Vj7fHDBLPEJdNvWj/view?usp=drive_link

buildStats:

```
rino@LAPTOP-ESI19P0R:/mnt/d/Tucil2rino@LAPTOP-ESI19P0R:/mnt/d/Tucil2_13524138_13524137/src$ go run /mnt/d/Tucil2_13524138_1352
4137/src/main.go  /mnt/d/Tucil2_13524138_13524137/test/konten.obj 5
Voxels (filled leaf nodes) : 286
Vertices in output         : 2288
Faces in output            : 1716

Octree node stats (nodes built per depth):
  1 : 1
  2 : 8
  3 : 56
  4 : 120
  5 : 264
  6 : 736

  3 : 41
  4 : 87
  5 : 172
  6 : 450

Octree depth               : 5
```

BAB 4 LAMPIRAN

4.1. Link Repository Program

Tautan Github:
https://github.com/BourneAgainSpicy/Tucil2_13524138_13524114.git

4.2. Pernyataan Tidak Melakukan Kecurangan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Mirza Tsabita Wafa'Ana

Ahmad Rinofaros Muchtar

Daftar Pustaka

1. https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/07-Algoritma-Divide-and-Conquer-(2026)-Bagian1.pdf
2. https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/07-Algoritma-Divide-and-Conquer-(2026)-Bagian2.pdf
3. https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/07-Algoritma-Divide-and-Conquer-(2026)-Bagian3.pdf
4. https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/Tucil2-IF2211-2026.pdf
5. https://go.dev/